# DEOS Network & System Management ICD

Sürüm 1.2 – SYSTEM Servisi, Ağ Yönetimi ve Düğüm Bilgisi

**Emir Arkalı** (emirarkali07@gmail.com)
DEOS – Dokuz Eylül Otonom Sistemler

İlk Yayın: 28 Eylül 2026 | Son Revizyon: 15 Ekim 2026

| | | | |
|---|---|---|---|
| **Doküman Tipi:** | Interface Control Document (ICD) | **Doküman Kodu:** | `DOC-ICD-NET-003` |
| **Doküman Sürümü:** | Sürüm 1.2 | **Protokol Sürümü:** | v1.1 (`0x11`) – Medya Bağımsız |
| **Yazar:** | Emir Arkalı (`emirarkali07@gmail.com`) | **Son Değiştiren:** | Emir Arkalı |
| **İlk Yayın Tarihi:** | 28 Eylül 2026 | **Son Revizyon Tarihi:** | 15 Ekim 2026 |
| **Durum:** | Yürürlükte / Onaylandı | **Taşıma / Kapsam:** | SYSTEM Servisi (`0x00`), PING/PONG & Düğüm Bilgisi |

### Doküman Revizyon Geçmişi

| Revizyon | Tarih | Yazar / Değiştiren | Açıklama |
|---|---|---|---|
| v0.3 | 28 Eylül 2026 | Emir Arkalı | İlk taslak ve monolitik ICD v0.3 tanımı. |
| v1.0 | 04 Ekim 2026 | Emir Arkalı | Ağ yönetimi bağımsız doküman oluşturuldu. |
| v1.1 | 13 Ekim 2026 | Emir Arkalı | Sürüm 1.1 standardizasyonu, kurumsal üstveri ve revizyon geçmişi eklendi. |
| v1.2 | 15 Ekim 2026 | Emir Arkalı | Düğüm bilgisi sorgulama (GET_NODE_INFO) istek ve 20 baytlık standart cevap formatı eklendi. |

## 1 SYSTEM Service Komutları

| ID | Symbol |
|---|---|
| 0x01 | DEOS_CMD_SYSTEM_GET_STATE |
| 0x02 | DEOS_CMD_SYSTEM_SET_STATE |
| 0x03 | DEOS_CMD_SYSTEM_GET_MODE |
| 0x04 | DEOS_CMD_SYSTEM_SET_MODE |
| 0x05 | DEOS_CMD_SYSTEM_RESET_NODE |
| 0x06 | DEOS_CMD_SYSTEM_GET_NODE_INFO |
| 0x07 | DEOS_CMD_SYSTEM_HEARTBEAT |
| 0x08 | DEOS_CMD_SYSTEM_PING |
| 0x09 | DEOS_CMD_SYSTEM_PONG |

```
typedef enum
{
    DEOS_CMD_SYSTEM_GET_STATE     = 0x01,
    DEOS_CMD_SYSTEM_SET_STATE     = 0x02,
    DEOS_CMD_SYSTEM_GET_MODE      = 0x03,
    DEOS_CMD_SYSTEM_SET_MODE      = 0x04,
    DEOS_CMD_SYSTEM_RESET_NODE    = 0x05,
    DEOS_CMD_SYSTEM_GET_NODE_INFO = 0x06,
```

```
    DEOS_CMD_SYSTEM_HEARTBEAT      = 0x07,
    DEOS_CMD_SYSTEM_PING           = 0x08,
    DEOS_CMD_SYSTEM_PONG           = 0x09
} deos_system_command_t;
```

# 2 PING–PONG Ağ Erişilebilirlik Mekanizması

DEOS protokolünde tüm adreslenebilir physical ve virtual node'lar ortak bir PING–PONG mekanizması uygular. Bir node, erişmek istediği başka bir node'a unicast PING gönderebilir. Geçerli PING mesajını alan hedef node, aynı Ping ID değerini içeren PONG mesajını kaynak node'a geri göndermek zorundadır.

PING ve PONG uygulama REQUEST/RESPONSE mekanizmasından ayrı bir network-management mekanizmasıdır. Her iki mesaj da aşağıdaki alanları kullanır:

```
Priority      = DEOS_PRIO_NETWORK
Message Class = DEOS_CLASS_NETWORK
Service       = DEOS_SERVICE_SYSTEM
```

PING için `Command = DEOS_CMD_SYSTEM_PING`; PONG için `Command = DEOS_CMD_SYSTEM_PONG` kullanılır.

## 2.1 PING Mesaj Formatı

PING mesajının uygulama veri alanı aşağıdaki gibi tanımlanır:

| Byte | Alan | Tip | Açıklama |
|---|---|---|---|
| 0 | Protocol Version | `uint8_t` | DEOS v1.1 için `0x11` |
| 1 | Sequence | `uint8_t` | Gönderen node'un TX sequence değeri |
| 2 | Command | `uint8_t` | `DEOS_CMD_SYSTEM_PING` = `0x08` |
| 3 | Length | `uint8_t` | `0x04` |
| 4–7 | Ping ID | `uint32_t` | Gönderen tarafından üretilen correlation değeri |

Toplam semantic CAN-FD data length 8 byte'tır. Ping ID, DEOS genel byte-order kuralına uygun olarak Little-Endian taşınır. Gönderen node, aynı anda birden fazla PING işlemi yürütüyorsa aktif işlemleri ayırt edecek Ping ID değerleri kullanmalıdır.

PING CAN Identifier alanları:

```
Priority      = DEOS_PRIO_NETWORK
Message Class = DEOS_CLASS_NETWORK
Service       = DEOS_SERVICE_SYSTEM
Destination   = ping atılan node
Source        = ping gönderen node
```

## 2.2 PONG Mesaj Formatı

PONG mesajı, alınan PING mesajındaki 32-bit Ping ID değerini değiştirmeden geri taşır.

| Byte | Alan | Tip | Açıklama |
|---|---|---|---|
| 0 | Protocol Version | `uint8_t` | DEOS v1.1 için `0x11` |
| 1 | Sequence | `uint8_t` | PONG gönderen node'un kendi TX sequence değeri |
| 2 | Command | `uint8_t` | `DEOS_CMD_SYSTEM_PONG` = `0x09` |
| 3 | Length | `uint8_t` | `0x04` |
| 4–7 | Ping ID | `uint32_t` | PING'den aynen echo edilen correlation değeri |

PONG CAN Identifier alanları:

```
Priority      = DEOS_PRIO_NETWORK
Message Class = DEOS_CLASS_NETWORK
Service       = DEOS_SERVICE_SYSTEM
Destination   = PING mesajının Source Node değeri
Source        = PING'i alan hedef node
```

PONG mesajındaki Sequence alanı PING mesajındaki Sequence değerini kopyalamaz. Her node kendi transmit sequence sayacını kullanmaya devam eder. PING ile PONG arasındaki transaction correlation yalnızca Ping ID üzerinden yapılır.

## 2.3 Zorunlu Node Davranışı

Geçerli ve kendisine yöneltilmiş bir PING alan her DEOS node'u aşağıdaki davranışı uygular:

1. PING mesajının Source Node ve Ping ID alanlarını okur.
2. Yeni bir PONG mesajı oluşturur.
3. PONG Source alanını kendi Node ID'si yapar.
4. PONG Destination alanını PING Source değerine eşitler.
5. PING içerisindeki Ping ID değerini değiştirmeden PONG payload'ına kopyalar.
6. Kendi normal TX Sequence değerini kullanarak PONG'u gönderir.

## 2.4 RTT Ölçümü

Round-trip time ölçümü gönderen node üzerinde yapılır. PING gönderildiği anda yerel zaman kaydedilir; aynı Ping ID'yi taşıyan PONG alındığında iki yerel zaman arasındaki fark RTT olarak hesaplanır:

$$RTT = T_{pong} - T_{ping}$$

Bu yöntem node'lar arasında saat senkronizasyonu gerektirmez. Timeout süresi transport veya uygulama gereksinimine göre ayrıca tanımlanabilir; v0.3 kapsamında tek bir global timeout süresi dondurulmamıştır.

## 2.5 Broadcast PING Kuralı

`DEOS_CMD_SYSTEM_PING` mesajı v1.1 protokolünde yalnızca unicast Destination Node ile kullanılmalıdır. `Destination = DEOS_NODE_GLOBAL_BROADCAST` veya `DEOS_NODE_CAN_BROADCAST` ile PING gönderimi geçersizdir. Bu kural, çok sayıda node'un aynı anda PONG üretmesinden doğacak response burst trafiğini önlemek için tanımlanmıştır.

Bir uygulama tüm node'ların erişilebilirliğini taramak istiyorsa her hedef Node ID için ayrı unicast PING işlemi başlatmalıdır.

## 2.6 HEARTBEAT Mesaj Formatı

HEARTBEAT, bir node'un periyodik veya olay tabanlı olarak kendi yaşam bilgisini yayınlamasıdır. PING–PONG ise belirli bir node'un başka belirli bir node'a aktif erişilebilirlik sorgusu yapmasıdır. Bu iki mekanizma birbirinin yerine kullanılmaz.

v1.1 protokolünde HEARTBEAT mesajı boş (0 byte payload) değildir. Node'un anlık durumunu (State) taşıyan 1 byte'lık bir payload barındırır:

| Byte | Alan | Tip | Açıklama |
|---|---|---|---|
| 0 | Current State | `uint8_t` | `deos_state_t` enum değeri (örn: `0x03` ACTIVE, `0x05` SAFE) |

# 3 Düğüm Bilgisi (GET_NODE_INFO) Sorgulama Mekanizması

DEOS protokolünde yer alan bir düğümün donanım tipi, firmware revizyonu, protokol versiyonu, aktif çalışma süresi (uptime) ve desteklediği servis haritasını (capabilities bitmask) sorgulamak için `DEOS_CMD_SYSTEM_GET_NODE_INFO` (`0x06`) komutu kullanılır.

Bu mekanizma, özellikle telemetri bilgisayarı (Raspberry Pi / Textual UI), Yer Kontrol İstasyonu ve ROS 2 Köprüsü (`deos_ros_bridge`) tarafından ağa yeni bağlanan veya hatta bulunan düğümlerin durumunu ve sürüm uyumluluğunu doğrulamak için kullanılır.

## 3.1 İstek (REQUEST) Mesaj Formatı

Düğüm bilgisi sorgulama işlemi bir istek–cevap (REQUEST–RESPONSE) işlemidir. İstek mesajı aşağıdaki başlık alanlarını kullanır:

```
Priority      = DEOS_PRIO_NETWORK (0x05) veya DEOS_PRIO_LOW (0x06)
Message Class = DEOS_CLASS_REQUEST (0x04)
Service       = DEOS_SERVICE_SYSTEM (0x00)
Command       = DEOS_CMD_SYSTEM_GET_NODE_INFO (0x06)
```

| Byte | Alan | Tip | Açıklama |
|---|---|---|---|
| 0 | Protocol Version | `uint8_t` | DEOS v1.1 için `0x11` |
| 1 | Sequence | `uint8_t` | İstek yapan node'un TX sequence değeri |
| 2 | Command | `uint8_t` | `DEOS_CMD_SYSTEM_GET_NODE_INFO` = `0x06` |
| 3 | Length | `uint8_t` | `0x00` (Ek payload verisi bulunmaz) |

### 3.1.1 Unicast Zorunluluğu

Tıpkı PING mekanizmasında olduğu gibi, `DEOS_CMD_SYSTEM_GET_NODE_INFO` istekleri yalnızca **Unicast** hedef adreslerle (`Destination != DEOS_NODE_GLOBAL_BROADCAST` ve `Destination != DEOS_NODE_CAN_BROADCAST`) gönderilmelidir. Broadcast olarak atılan istekler, özellikle CAN-FD ve LoRa hattında aynı anda çok sayıda düğümün cevap üretmesine (response burst/collision) ve bant genişliğinin kilitlenmesine neden olacağı için kabul edilmez.

## 3.2 Cevap (RESPONSE) Mesaj Formatı

Sorgulanan hedef düğüm, geçerli bir istek aldığında `DEOS_CLASS_RESPONSE` sınıfında 20 baytlık standart bir cevap üretir.

Cevap başlık alanları:

```
Priority      = DEOS_PRIO_NETWORK (0x05)
Message Class = DEOS_CLASS_RESPONSE (0x05)
Service       = DEOS_SERVICE_SYSTEM (0x00)
Command       = DEOS_CMD_SYSTEM_GET_NODE_INFO (0x06)
Destination   = İstek gönderen kaynak node ID
Source        = Sorgulanan yerel node ID
```

Cevap veri yükü (Payload) anatomisi (Toplam 20 Byte, Little-Endian):

| Byte | Alan Adı | Tip | Boyut | Açıklama |
|---|---|---|---|---|
| 0 | Result Code | `uint8_t` | 1 Byte | İşlem sonucu: `0x00` SUCCESS (`deos_result_t`) |
| 1 | Node ID | `uint8_t` | 1 Byte | Cevap veren düğümün mantıksal kimliği (`deos_node_id_t`) |
| 2 | Hardware Type | `uint8_t` | 1 Byte | Donanım tipi (`0x01` STM32H5, `0x02` Linux/Pi, `0x03` ESP32) |
| 3 | Protocol Version | `uint8_t` | 1 Byte | Desteklenen wire format sürümü (`0x11` = v1.1) |
| 4 | FW Major | `uint8_t` | 1 Byte | Firmware ana sürümü (Örn: 1) |
| 5 | FW Minor | `uint8_t` | 1 Byte | Firmware alt sürümü (Örn: 2) |
| 6 | FW Patch | `uint8_t` | 1 Byte | Firmware yama sürümü (Örn: 0) |
| 7 | Reserved | `uint8_t` | 1 Byte | 4-bayt hizalama için dolgu baytı (`0x00`) |
| 8–11 | Git Hash | `uint32_t` | 4 Byte | Derlemenin Git commit hash'inin ilk 4 baytı |
| 12–15 | Supported Services | `uint32_t` | 4 Byte | Desteklenen servislerin bitmask değeri |
| 16–19 | Uptime | `uint32_t` | 4 Byte | Düğümün açılışından bu yana geçen süre (Saniye) |

### 3.3 Cevap Veri Yapısı (C Struct Tanımı)

```
typedef struct
{
    uint8_t  result_code;        /* 0x00: SUCCESS (deos_result_t) */
    uint8_t  node_id;            /* Logical Node ID (deos_node_id_t) */
    uint8_t  hw_type;            /* Hardware Type (e.g. 0x01 STM32H5) */
    uint8_t  proto_version;      /* Protocol Version (0x11 = v1.1) */
    uint8_t  fw_version_major;   /* Firmware Version - Major */
    uint8_t  fw_version_minor;   /* Firmware Version - Minor */
    uint8_t  fw_version_patch;   /* Firmware Version - Patch */
    uint8_t  reserved;           /* Alignment padding (0x00) */
    uint32_t git_hash;           /* Git commit hash prefix (4 bytes) */
    uint32_t supported_services; /* Bitmask: (1 << DEOS_SERVICE_SYSTEM) | ... */
    uint32_t uptime_s;           /* Node uptime in seconds */
} __attribute__((packed)) deos_node_info_payload_t;
```

### 3.4 Supported Services Bitmask Alanı

`supported_services` 32-bit bitmask alanı, sorgulanan düğümün hangi fonksiyonel servisleri barındırdığını belirtir:

| Bit | Servis Karşılığı | Değer |
|---|---|---|
| Bit 0 | `DEOS_SERVICE_SYSTEM` | `1 « 0x00` (0x00000001) |
| Bit 1 | `DEOS_SERVICE_TRACTION` | `1 « 0x01` (0x00000002) |
| Bit 2 | `DEOS_SERVICE_STEERING` | `1 « 0x02` (0x00000004) |
| Bit 3 | `DEOS_SERVICE_BRAKE` | `1 « 0x03` (0x00000008) |
| Bit 4 | `DEOS_SERVICE_BMS` | `1 « 0x04` (0x00000010) |
| Bit 5 | `DEOS_SERVICE_CALIBRATION` | `1 « 0x05` (0x00000020) |
| Bit 6 | `DEOS_SERVICE_DIAGNOSTIC` | `1 « 0x06` (0x00000040) |
| Bit 7 | `DEOS_SERVICE_SENSORS` | `1 « 0x07` (0x00000080) |
| Bit 8 | `DEOS_SERVICE_LIGHTING` | `1 « 0x08` (0x00000100) |